Experiment 6: Image Erosion

Aim: To read an input image using Python OpenCV and apply the erosion morphological operation using the cv2.erode() function.

Procedures

1. Import the required cv2 and numpy libraries.
2. Load the input image from the specified path using cv2.imread().
3. Define a 5×5 structurinsg element (kernel) composed of ones using np.ones().
4. Apply morphological erosion on the image using cv2.erode().
5. Display the original and eroded images using cv2.imshow().
6. Wait for user input and close all display windows using cv2.destroyAllWindows().

Code

```
import cv2

import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")

kernel = np.ones((5, 5), np.uint8)

eroded = cv2.erode(img, kernel, iterations=1)

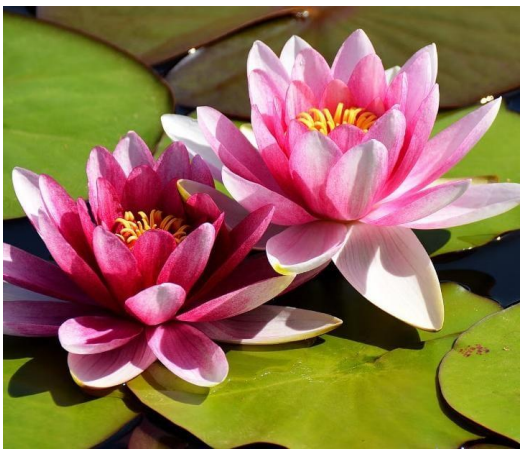
cv2.imshow("Original Image", img)

cv2.imshow("Eroded Image", eroded)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Input



Output



Result: The input image was successfully loaded and eroded using OpenCV's erosion function.

Experiment 7: Video Slow Motion and Fast Motion

Aim: To perform basic video processing operations by reading a video file and displaying it in slow motion and fast motion.

Procedure

1. Import the OpenCV library (cv2).
2. Load the video source using cv2.VideoCapture().
3. Read individual frames inside a continuous loop using cap.read().
4. Adjust the frame display delay with cv2.waitKey() to control speed (higher delay for slow motion, lower delay for fast motion).
5. Display each frame in a output window using cv2.imshow().
6. Release the video capture object and close all active windows when playback completes.

Code

```
import cv2

cap = cv2.VideoCapture(r"C:\Users\acer\Downloads\pretty lotus.jpeg")

while cap.isOpened():

    ret, frame = cap.read()

    if not ret:

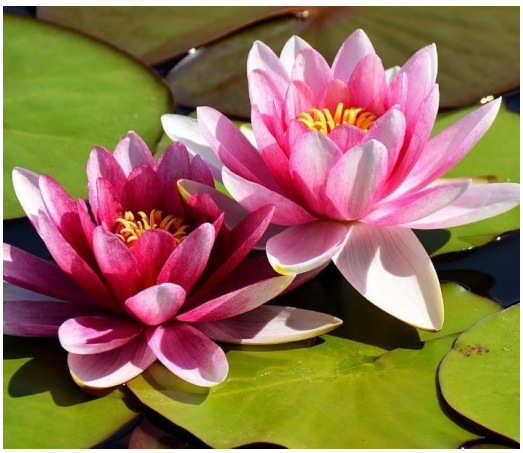
        break

    cv2.imshow("Video Display", frame)

    if cv2.waitKey(100) & 0xFF == ord('q'):
```

```
        break
cap.release()
cv2.destroyAllWindows()
```

Input



Output



Result: The video stream was successfully read and displayed with variable playback speed control using Python and OpenCV.

Experiment 8: Image Dilation

Aim: To read an input image using Python OpenCV and dilate the image using the cv2.dilate() function.

Procedure

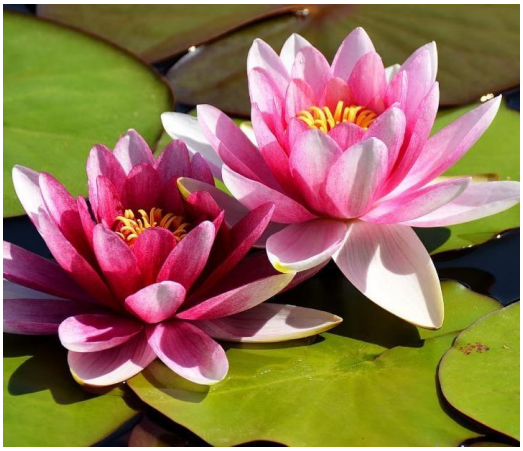
1. Import cv2 and numpy modules into Python.
2. Read the source image from the given file path using cv2.imread().
3. Construct a 5×5 kernel matrix filled with unsigned integers.
4. Apply the dilation operation on the image using cv2.dilate().
5. Show both original and dilated images using cv2.imshow().

6. Terminate display windows after key press using cv2.destroyAllWindows().

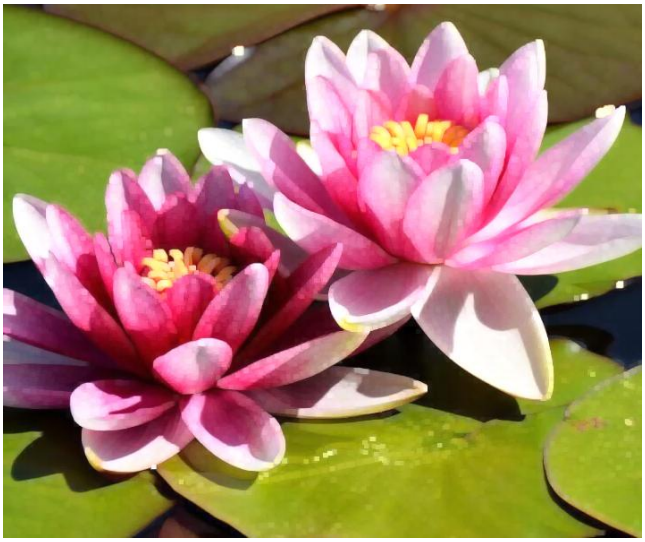
Code

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
kernel = np.ones((5, 5), np.uint8)
dilated = cv2.dilate(img, kernel, iterations=1)
cv2.imshow("Original Image", img)
cv2.imshow("Dilated Image", dilated)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The input image was successfully loaded and expanded using OpenCV's dilation function.

Experiment 9: Image Scaling (Resizing)

Aim: To implement image scaling techniques to resize an image into both bigger and smaller dimensions using OpenCV.

Procedure

1. Import the OpenCV library (cv2).
2. Load the input image from the specified file path using cv2.imread().
3. Define scale factors or target dimensions for downsizing and upsizing.
4. Scale down the image using cv2.resize() with fractional scaling factors.
5. Scale up the image using cv2.resize() with enlarged scaling factors.
6. Display the resized images in separate windows and release memory on key press.

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")

smaller = cv2.resize(img, (0, 0), fx=0.5, fy=0.5)

bigger = cv2.resize(img, (0, 0), fx=1.5, fy=1.5)

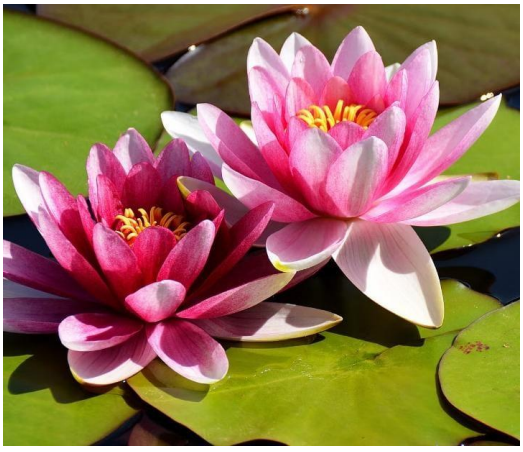
cv2.imshow("Smaller Image", smaller)

cv2.imshow("Bigger Image", bigger)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Input



Output





Result: The input image was successfully scaled and resized into both smaller and larger output dimensions.

Experiment 10: 90-Degree Rotation Clockwise

Aim: To perform a 90-degree clockwise rotation on the given input image using Python and OpenCV.

Procedure

1. Import the cv2 library.
2. Read the source image from the given file path.
3. Apply 90-degree clockwise rotation using cv2.rotate() with flag cv2.ROTATE_90_CLOCKWISE.
4. Display the original and rotated images in separate output windows.
5. Pause execution using cv2.waitKey(0) until a key is pressed.
6. Destroy all OpenCV display windows using cv2.destroyAllWindows().

Code

```
import cv2
img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
rotated90 = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
cv2.imshow("Original Image", img)
cv2.imshow("90 Degree Rotated", rotated90)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The given image was successfully rotated 90 degrees clockwise using Python OpenCV.